Приложение Б

Листинг программы

```
//Класс TourAgent.cs:
using System;
using System.Collections.Generic;
using System.Linq;
```

```
using System.Text;
using System.Threading.Tasks;

namespace TouristVoucher
{
   public class TourAgent
   {
      public string FullName { get; set; }
      public string InsuranceNumber { get; set; }
      public string INN { get; set; }
      public string PassportNumber { get; set; }
      public DateTime VisaExpiry { get; set; }

      // Для отображения в DataGrid
      public string VisaExpiryText =>
VisaExpiry.ToString("dd.MM.yyyy");
      public bool IsVisaValid => VisaExpiry >
DateTime.Now;
      public string VisaStatus => IsVisaValid ?
"Действительна" : "Просрочена";
   }
}
//класс TourGroup.cs
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TouristVoucher
{
   public class TourGroup
   {
      public string GroupNumber { get; set; }
      public int PeopleCount { get; set; }
      public string AgeCategory { get; set; }

      // Для отображения в DataGrid
      public string PeopleCountText =>
$"{PeopleCount} чел.";
   }
}
//класс TripSchedule
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TouristVoucher
{
   public class TripSchedule
   {
      public string AgentName { get; set; }
      public string GroupNumber { get; set; }
      public DateTime TripDate { get; set; }
      public string ContractNumber { get; set; }
      public int TripDuration { get; set; }

      // Для отображения в DataGrid
      public string TripDateText =>
TripDate.ToString("dd.MM.yyyy");
      public string ReturnDateText =>
TripDate.AddDays(TripDuration).ToString("dd.MM.
yyyy");
      public string TripDurationText =>
$"{TripDuration} дней";
   }
}
//класс TripInfo
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TouristVoucher
{
   public class TripInfo
   {
      // Информация о поездке
      public string AgentName { get; set; }
      public string GroupNumber { get; set; }
      public DateTime TripDate { get; set; }
      public string ContractNumber { get; set; }
      public int TripDuration { get; set; }

      // Информация о группе
      public int PeopleCount { get; set; }
      public string AgeCategory { get; set; }

      // Информация о турагенте
      public string InsuranceNumber { get; set; }
      public string INN { get; set; }
      public string PassportNumber { get; set; }
      public DateTime VisaExpiry { get; set; }

      // Для отображения в DataGrid (вычисляемые
свойства)
      public string TripDateText =>
TripDate.ToString("dd.MM.yyyy");
      public string ReturnDateText =>
TripDate.AddDays(TripDuration).ToString("dd.MM.
yyyy");
      public string VisaExpiryText =>
VisaExpiry.ToString("dd.MM.yyyy");
      public string VisaStatus => VisaExpiry >
DateTime.Now ? "✓ Действительна" : "✗
Просрочена";
      public string TripPeriod => $"{TripDateText} -
{ReturnDateText}";
   }
}
//MainWindow.xaml.cs
using System;
using System.Linq;
using System.Windows;
using System.Windows.Media;
using System.Xml.Linq;
using System.Collections.Generic;
using System.IO;
using System.Windows.Controls;

namespace TouristVoucher
{
```

```
    public partial class MainWindow : Window
    {
        // Списки данных (сделаем публичными для
доступа из других окон)
        public List<TourAgent> agents = new
List<TourAgent>();
        public List<TourGroup> groups = new
List<TourGroup>();
        public List<TripSchedule> schedule = new
List<TripSchedule>();
        private List<TripInfo> allTrips = new
List<TripInfo>();

        // Путь к XML файлу
        private string xmlFilePath;

        public MainWindow()
        {
            InitializeComponent();

            // Определяем путь к XML файлу
            xmlFilePath =
System.IO.Path.Combine(AppDomain.CurrentDomai
n.BaseDirectory, "TourData.xml");

            // Если файл не найден в папке Debug,
ищем в корне проекта
            if (!File.Exists(xmlFilePath))
            {
                string projectPath =
Directory.GetParent(AppDomain.CurrentDomain.Bas
eDirectory).Parent.Parent.FullName;
                xmlFilePath =
System.IO.Path.Combine(projectPath,
"TourData.xml");
            }
        }

        // Кнопка "Загрузить данные"
        private void btnLoad_Click(object sender,
RoutedEventArgs e)
        {
            try
            {
                LoadXmlData();
                txtStatus.Text = $"XML загружен.
Загружено {agents.Count} агентов, {groups.Count}
групп, {schedule.Count} поездок.";

                // Активируем все кнопки
                btnShowAll.IsEnabled = true;
                btnShowUpcoming.IsEnabled = true;
                btnShowPast.IsEnabled = true;
                btnCheckVisa.IsEnabled = true;
                btnReset.IsEnabled = true;
                btnLoad.IsEnabled = false;

                // Активируем новые кнопки управления
                btnManageAgents.IsEnabled = true;
                btnManageGroups.IsEnabled = true;
                btnManageTrips.IsEnabled = true;
                btnExportCSV.IsEnabled = true;
                btnShowChart.IsEnabled = true;

                // Автоматически показываем все
поездки
                ShowAllTrips();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Ошибка загрузки
XML: {ex.Message}", "Ошибка",
                    MessageBoxButton.OK,
MessageBoxImage.Error);
                txtStatus.Text = "Ошибка загрузки XML";
            }
        }

        // Загрузка данных из XML
        private void LoadXmlData()
        {
            if (!File.Exists(xmlFilePath))
            {
                throw new
FileNotFoundException($"Файл {xmlFilePath} не
найден");
            }
            XDocument doc =
XDocument.Load(xmlFilePath);

            // Загрузка турагентов
            agents = (from a in
doc.Root.Element("Agents").Elements("Agent")
                    select new TourAgent
                    {
                        FullName =
(string)a.Attribute("fullName"),
                        InsuranceNumber =
(string)a.Attribute("insuranceNumber"),
                        INN = (string)a.Attribute("inn"),
                        PassportNumber =
(string)a.Attribute("passport"),
                        VisaExpiry =
DateTime.Parse((string)a.Attribute("visaExpiry"))
                    }).ToList();

            // Загрузка групп
            groups = (from g in
doc.Root.Element("Groups").Elements("Group")
                    select new TourGroup
                    {
                        GroupNumber =
(string)g.Attribute("number"),
                        PeopleCount =
(int)g.Attribute("peopleCount"),
                        AgeCategory =
(string)g.Attribute("ageCategory")
                    }).ToList();

            // Загрузка графика поездок
            schedule = (from t in
doc.Root.Element("Schedule").Elements("Trip")
                    select new TripSchedule
                    {
                        AgentName =
(string)t.Attribute("agentName"),
```

```
                    GroupNumber =
(string)t.Attribute("groupNumber"),
                    TripDate =
DateTime.Parse((string)t.Attribute("date")),
                    ContractNumber =
(string)t.Attribute("contractNumber"),
                    TripDuration =
(int)t.Attribute("tripDuration")
                 }).ToList();

        UpdateAllTrips();
    }

    // Обновление объединенного списка поездок
    public void UpdateAllTrips()
    {
        allTrips = new List<TripInfo>();

        foreach (var t in schedule)
        {
            // Находим агента по имени
            TourAgent agent = null;
            foreach (var a in agents)
            {
                if (a.FullName == t.AgentName)
                {
                    agent = a;
                    break;
                }
            }

            // Находим группу по номеру
            TourGroup group = null;
            foreach (var g in groups)
            {
                if (g.GroupNumber == t.GroupNumber)
                {
                    group = g;
                    break;
                }
            }

            // Если агент и группа найдены,
добавляем поездку
            if (agent != null && group != null)
            {
                TripInfo trip = new TripInfo();
                trip.AgentName = t.AgentName;
                trip.GroupNumber = t.GroupNumber;
                trip.TripDate = t.TripDate;
                trip.ContractNumber =
t.ContractNumber;
                trip.TripDuration = t.TripDuration;
                trip.PeopleCount = group.PeopleCount;
                trip.AgeCategory = group.AgeCategory;
                trip.InsuranceNumber =
agent.InsuranceNumber;
                trip.INN = agent.INN;
                trip.PassportNumber =
agent.PassportNumber;
                trip.VisaExpiry = agent.VisaExpiry;

                allTrips.Add(trip);
            }
        }

        // Сортируем по дате
        allTrips = allTrips.OrderBy(t =>
t.TripDate).ToList();

        // Обновляем отображение, если
показываем все поездки
        if (dgTrips.ItemsSource != null)
        {
            ShowAllTrips();
        }

        UpdateStatistics();
    }

    // Сохранение данных в XML
    public void SaveAllData()
    {
        try
        {
            XDocument doc = new XDocument();
            XElement root = new
XElement("TourCompany");

            // Сохраняем агентов
            XElement agentsElement = new
XElement("Agents");
            foreach (var agent in agents)
            {
                XElement agentElement = new
XElement("Agent",
                    new XAttribute("fullName",
agent.FullName),
                    new XAttribute("insuranceNumber",
agent.InsuranceNumber),
                    new XAttribute("inn", agent.INN),
                    new XAttribute("passport",
agent.PassportNumber),
                    new XAttribute("visaExpiry",
agent.VisaExpiry.ToString("yyyy-MM-dd")));
                agentsElement.Add(agentElement);
            }
            root.Add(agentsElement);

            // Сохраняем группы
            XElement groupsElement = new
XElement("Groups");
            foreach (var group in groups)
            {
                XElement groupElement = new
XElement("Group",
                    new XAttribute("number",
group.GroupNumber),
                    new XAttribute("peopleCount",
group.PeopleCount),
                    new XAttribute("ageCategory",
group.AgeCategory));
                groupsElement.Add(groupElement);
            }
            root.Add(groupsElement);
```

```
            // Сохраняем график поездок
            XElement scheduleElement = new
XElement("Schedule");
            foreach (var trip in schedule)
            {
                XElement tripElement = new
XElement("Trip",
                    new XAttribute("agentName",
trip.AgentName),
                    new XAttribute("groupNumber",
trip.GroupNumber),
                    new XAttribute("date",
trip.TripDate.ToString("yyyy-MM-dd")),
                    new XAttribute("contractNumber",
trip.ContractNumber),
                    new XAttribute("tripDuration",
trip.TripDuration));
                scheduleElement.Add(tripElement);
            }
            root.Add(scheduleElement);

            doc.Add(root);
            doc.Save(xmlFilePath);

            txtStatus.Text = "Данные сохранены в
XML";
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка сохранения
XML: {ex.Message}", "Ошибка",
                MessageBoxButton.OK,
MessageBoxImage.Error);
        }
    }

    // Обновление всех данных (после
изменений)
    public void UpdateAllData()
    {
        UpdateAllTrips();
        UpdateStatistics();
        txtStatus.Text = "Данные обновлены";
    }

    // Показать все поездки
    private void ShowAllTrips()
    {
        dgTrips.ItemsSource = allTrips;
        UpdateStatistics();
        txtStatus.Text = $"Показано {allTrips.Count}
поездок";
    }

    // Кнопка "Все поездки"
    private void btnShowAll_Click(object sender,
RoutedEventArgs e)
    {
        ShowAllTrips();
    }

    // Кнопка "Предстоящие"
    private void btnShowUpcoming_Click(object
sender, RoutedEventArgs e)
    {
        var upcomingTrips = allTrips.Where(t =>
t.TripDate >= DateTime.Now.Date).ToList();
        dgTrips.ItemsSource = upcomingTrips;
        UpdateStatistics();
        txtStatus.Text = $"Показано
{upcomingTrips.Count} предстоящих поездок";
    }

    // Кнопка "Прошедшие"
    private void btnShowPast_Click(object sender,
RoutedEventArgs e)
    {
        var pastTrips = allTrips.Where(t => t.TripDate
< DateTime.Now.Date).ToList();
        dgTrips.ItemsSource = pastTrips;
        UpdateStatistics();
        txtStatus.Text = $"Показано
{pastTrips.Count} прошедших поездок";
    }

    // Кнопка "Проверить визы"
    private void btnCheckVisa_Click(object sender,
RoutedEventArgs e)
    {
        var expiredAgents = agents.Where(a =>
a.VisaExpiry < DateTime.Now).ToList();
        var expiringSoon = agents.Where(a =>
a.VisaExpiry >= DateTime.Now && a.VisaExpiry <=
DateTime.Now.AddDays(30)).ToList();

        string message = "=== ПРОВЕРКА ВИЗ
===\n\n";

        if (expiredAgents.Any())
        {
            message += "❌ АГЕНТЫ С
ПРОСРОЧЕННОЙ ВИЗОЙ:\n";
            foreach (var agent in expiredAgents)
            {
                message += $"  - {agent.FullName}
(виза до {agent.VisaExpiry:dd.MM.yyyy})\n";
            }
            message += "\n";
        }
        else
        {
            message += "✓ Нет агентов с
просроченной визой.\n\n";
        }

        if (expiringSoon.Any())
        {
            message += "⚠️ ВИЗЫ ИСТЕКАЮТ В
БЛИЖАЙШИЕ 30 ДНЕЙ:\n";
            foreach (var agent in expiringSoon)
            {
                int daysLeft = (agent.VisaExpiry -
DateTime.Now).Days;
```

```
                message += $"   - {agent.FullName}
(истекает через {daysLeft} дней,
{agent.VisaExpiry:dd.MM.yyyy})\n";
            }
            message += "\n";
        }
        else
        {
            message += "✓ Нет виз, истекающих в
ближайшие 30 дней.\n\n";
        }

        message += $"Всего агентов:
{agents.Count}\n";
        message += $"Действительных виз:
{agents.Count(a => a.VisaExpiry >=
DateTime.Now)}\n";
        message += $"Просроченных виз:
{expiredAgents.Count}";

        MessageBox.Show(message, "Результаты
проверки виз", MessageBoxButton.OK,
            expiredAgents.Any() ?
MessageBoxImage.Warning :
MessageBoxImage.Information);

        txtStatus.Text = $"Проверка виз завершена.
Агентов с просрочкой: {expiredAgents.Count}";

        UpdateStatistics();
    }

    // Кнопка "Управление агентами"
    private void btnManageAgents_Click(object
sender, RoutedEventArgs e)
    {
        AgentsWindow agentsWindow = new
AgentsWindow(agents, this);
        agentsWindow.Owner = this;
        agentsWindow.ShowDialog();
    }

    // Кнопка "Управление группами"
    private void btnManageGroups_Click(object
sender, RoutedEventArgs e)
    {
        GroupsWindow groupsWindow = new
GroupsWindow(groups, this);
        groupsWindow.Owner = this;
        groupsWindow.ShowDialog();
    }

    // Кнопка "Управление поездками"
    private void btnManageTrips_Click(object
sender, RoutedEventArgs e)
    {
        TripsWindow tripsWindow = new
TripsWindow(schedule, agents, groups, this);
        tripsWindow.Owner = this;
        tripsWindow.ShowDialog();
    }

    // Кнопка "Экспорт в CSV"
    private void btnExportCSV_Click(object sender,
RoutedEventArgs e)
    {
        try
        {
            // Получаем текущий список поездок из
DataGrid
            var currentTrips = dgTrips.ItemsSource as
List<TripInfo>;
            if (currentTrips == null ||
currentTrips.Count == 0)
            {
                MessageBox.Show("Нет данных для
экспорта", "Внимание",
                    MessageBoxButton.OK,
MessageBoxImage.Warning);
                return;
            }

            // Диалог сохранения файла
            Microsoft.Win32.SaveFileDialog
saveDialog = new
Microsoft.Win32.SaveFileDialog();
            saveDialog.Filter = "CSV файлы
(*.csv)|*.csv";
            saveDialog.DefaultExt = ".csv";
            saveDialog.FileName =
$"Поездки_{DateTime.Now:yyyyMMdd_HHmmss}
";

            if (saveDialog.ShowDialog() == true)
            {
                using (StreamWriter writer = new
StreamWriter(saveDialog.FileName, false,
System.Text.Encoding.UTF8))
                {
                    // Заголовки
                    writer.WriteLine("Турагент;Номер
группы;Дата поездки;Дата
возврата;Длительность;Номер
договора;Количество
человек;Категория;ИНН;Срок визы;Статус
визы");

                    // Данные
                    foreach (var trip in currentTrips)
                    {

writer.WriteLine($"\"{trip.AgentName}\";{trip.Grou
pNumber};{trip.TripDate:dd.MM.yyyy};{trip.Return
DateText};{trip.TripDuration};{trip.ContractNumber
};{trip.PeopleCount};{trip.AgeCategory};{trip.INN}
;{trip.VisaExpiryText};{trip.VisaStatus}");
                    }
                }

                txtStatus.Text = $"Экспортировано
{currentTrips.Count} поездок в
{saveDialog.FileName}";
                MessageBox.Show($"Данные успешно
экспортированы!\nФайл: {saveDialog.FileName}",
```

```
                "Экспорт завершен",
MessageBoxButton.OK,
MessageBoxImage.Information);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка экспорта:
{ex.Message}", "Ошибка",
                MessageBoxButton.OK,
MessageBoxImage.Error);
        }
    }

    // Выбор поездки из таблицы
    private void dgTrips_SelectionChanged(object
sender,
System.Windows.Controls.SelectionChangedEventAr
gs e)
    {
        if (dgTrips.SelectedItem is TripInfo
selectedTrip)
        {
            ShowTripDetails(selectedTrip);
            txtStatus.Text = $"Выбрана поездка:
{selectedTrip.GroupNumber} -
{selectedTrip.AgentName}";
        }
    }

    // Показать детальную информацию о
поездке
    private void ShowTripDetails(TripInfo trip)
    {
        txtTripAgent.Text = $"Турагент:
{trip.AgentName}";
        txtTripGroup.Text = $"Группа:
{trip.GroupNumber}";
        txtTripPeriod.Text = $"Период:
{trip.TripPeriod}";
        txtContract.Text = $"Договор:
{trip.ContractNumber}";

        txtGroupPeople.Text = $"Количество:
{trip.PeopleCount} чел.";
        txtGroupCategory.Text = $"Категория:
{trip.AgeCategory}";

        txtAgentInsurance.Text = $"Страховое:
{trip.InsuranceNumber}";
        txtAgentINN.Text = $"ИНН: {trip.INN}";
        txtAgentPassport.Text = $"Паспорт:
{trip.PassportNumber}";
        txtAgentVisa.Text = $"Срок визы:
{trip.VisaExpiryText}";
        txtAgentVisaStatus.Text = $"Статус визы:
{trip.VisaStatus}";

        if (trip.VisaExpiry < DateTime.Now)
        {
            txtAgentVisaStatus.Foreground =
Brushes.Red;
            txtAgentVisaStatus.FontWeight =
FontWeights.Bold;
        }
        else if (trip.VisaExpiry <=
DateTime.Now.AddDays(30))
        {
            txtAgentVisaStatus.Foreground =
Brushes.Orange;
            txtAgentVisaStatus.FontWeight =
FontWeights.Bold;
        }
        else
        {
            txtAgentVisaStatus.Foreground =
Brushes.Green;
            txtAgentVisaStatus.FontWeight =
FontWeights.Normal;
        }
    }

    // Обновление статистики
    private void UpdateStatistics()
    {
        txtTotalTrips.Text = $"Всего поездок:
{allTrips.Count}";

        int upcoming = allTrips.Count(t => t.TripDate
>= DateTime.Now.Date);
        txtUpcomingTrips.Text = $"Предстоящих:
{upcoming}";

        int past = allTrips.Count(t => t.TripDate <
DateTime.Now.Date);
        txtPastTrips.Text = $"Прошедших: {past}";

        int expiredVisas = agents.Count(a =>
a.VisaExpiry < DateTime.Now);
        txtVisaExpired.Text = $"Агентов с просроч.
визой: {expiredVisas}";

        if (expiredVisas > 0)
        {
            txtVisaExpired.Foreground = Brushes.Red;
        }
        else
        {
            txtVisaExpired.Foreground =
Brushes.Green;
        }
    }

    // Кнопка "Сбросить"
    private void btnReset_Click(object sender,
RoutedEventArgs e)
    {
        dgTrips.ItemsSource = null;
        ClearTripDetails();
        txtStatus.Text = "Данные сброшены.
Нажмите 'Загрузить данные' для новой загрузки.";

        btnLoad.IsEnabled = true;
        btnShowAll.IsEnabled = false;
        btnShowUpcoming.IsEnabled = false;
```

```
            btnShowPast.IsEnabled = false;
            btnCheckVisa.IsEnabled = false;
            btnReset.IsEnabled = false;
            btnManageAgents.IsEnabled = false;
            btnManageGroups.IsEnabled = false;
            btnManageTrips.IsEnabled = false;
            btnExportCSV.IsEnabled = false;
            btnShowChart.IsEnabled = false;

            txtTotalTrips.Text = "Всего поездок: -";
            txtUpcomingTrips.Text = "Предстоящих: -";
            txtPastTrips.Text = "Прошедших: -";
            txtVisaExpired.Text = "Агентов с просроч.
визой: -";
        }

        // Очистка детальной информации
        private void ClearTripDetails()
        {
            txtTripAgent.Text = "Турагент: -";
            txtTripGroup.Text = "Группа: -";
            txtTripPeriod.Text = "Период: -";
            txtContract.Text = "Договор: -";
            txtGroupPeople.Text = "Количество: -";
            txtGroupCategory.Text = "Категория: -";
            txtAgentInsurance.Text = "Страховое: -";
            txtAgentINN.Text = "ИНН: -";
            txtAgentPassport.Text = "Паспорт: -";
            txtAgentVisa.Text = "Срок визы: -";
            txtAgentVisaStatus.Text = "Статус визы: -";
        }

        // ========== МЕТОДЫ ПОИСКА И
ФИЛЬТРАЦИИ ==========

        // Кнопка "Поиск"
        private void btnSearch_Click(object sender,
RoutedEventArgs e)
        {
            var filteredTrips = allTrips.AsEnumerable();

            // Фильтр по агенту
            if
(!string.IsNullOrWhiteSpace(txtSearchAgent.Text))
            {
                string searchAgent =
txtSearchAgent.Text.ToLower();
                filteredTrips = filteredTrips.Where(t =>
t.AgentName.ToLower().Contains(searchAgent));
            }

            // Фильтр по группе
            if
(!string.IsNullOrWhiteSpace(txtSearchGroup.Text))
            {
                string searchGroup =
txtSearchGroup.Text.ToLower();
                filteredTrips = filteredTrips.Where(t =>
t.GroupNumber.ToLower().Contains(searchGroup));
            }

            // Фильтр по дате "с"
            if (dpDateFrom.SelectedDate.HasValue)
            {
                DateTime fromDate =
dpDateFrom.SelectedDate.Value.Date;
                filteredTrips = filteredTrips.Where(t =>
t.TripDate.Date >= fromDate);
            }

            // Фильтр по дате "по"
            if (dpDateTo.SelectedDate.HasValue)
            {
                DateTime toDate =
dpDateTo.SelectedDate.Value.Date;
                filteredTrips = filteredTrips.Where(t =>
t.TripDate.Date <= toDate);
            }

            var result = filteredTrips.ToList();
            dgTrips.ItemsSource = result;
            txtStatus.Text = $"Найдено {result.Count}
поездок";
        }

        // Кнопка "Сброс фильтра"
        private void btnClearSearch_Click(object
sender, RoutedEventArgs e)
        {
            // Очищаем поля поиска
            txtSearchAgent.Text = "";
            txtSearchGroup.Text = "";
            dpDateFrom.SelectedDate = null;
            dpDateTo.SelectedDate = null;

            // Показываем все поездки
            ShowAllTrips();
            txtStatus.Text = "Фильтр сброшен";
        }

        // ========== КНОПКА ДЛЯ ГРАФИКА
==========

        // Кнопка "График поездок"
        private void btnShowChart_Click(object sender,
RoutedEventArgs e)
        {
            if (allTrips.Count == 0)
            {
                MessageBox.Show("Нет данных для
отображения графика. Сначала загрузите
данные.",
                    "Информация",
MessageBoxButton.OK,
MessageBoxImage.Information);
                return;
            }

            var chartWindow = new
ChartWindow(allTrips);
            chartWindow.Owner = this;
            chartWindow.ShowDialog();
        }
    }
}
```